

Data Structures

Lecture 8: DLL & Dictionary

Soobin Um

Apr. 1, 2026

복습

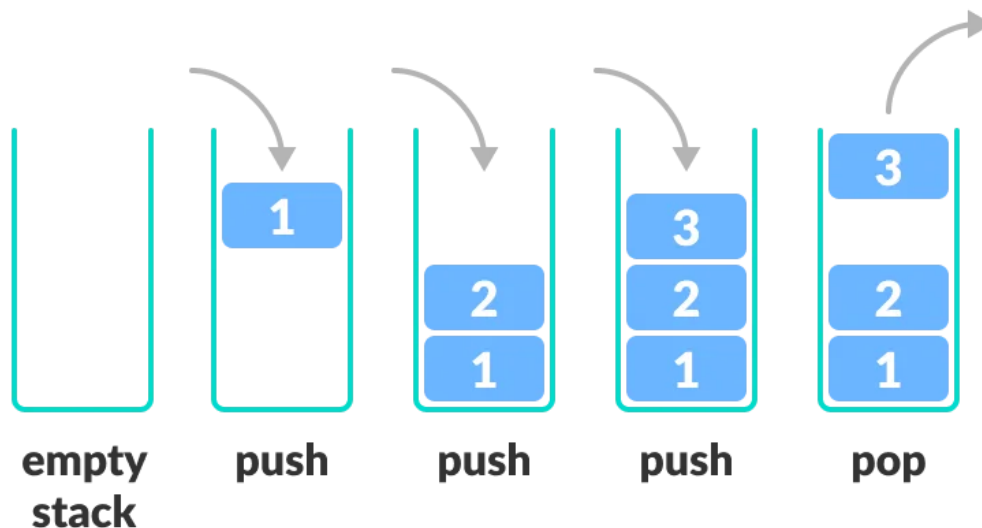
스택: 구조 및 주요 연산

- 구조

- 리스트의 한쪽 끝(top)에서만 삽입 또는 삭제 연산이 이뤄짐

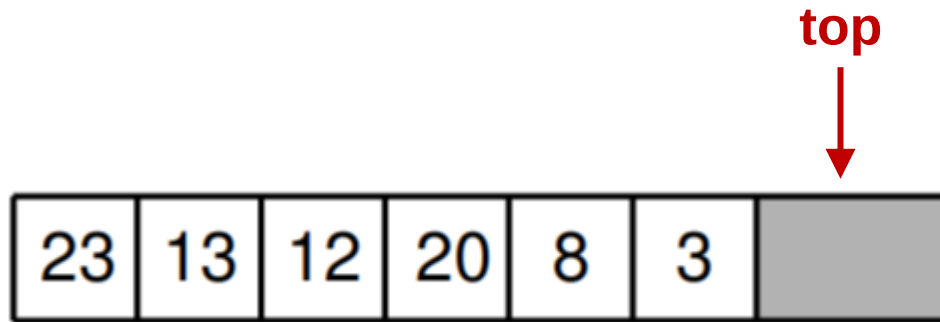
- 주요 연산

- **push(item)**: 스택의 맨 위에 원소(item)를 삽입
- **pop()**: 스택의 맨 위 원소를 꺼내어 제거
- **topValue()**: 스택의 맨 위 원소를 제거하지 않고 확인 (리턴)



ArrayStack: 구조

- 내부에 **고정된 크기의 배열**을 둔 상태에서,
- 인덱스 변수(top)를 통해 현재 스택의 **맨 위 위치를 추적**



ArrayStack: 코드

```
public class ArrayStack<E> implements Stack<E> {
    private E[] listArray;    // 스택 원소 저장
    private int top;          // 스택 맨 위 인덱스
    private int capacity;

    public ArrayStack(int size) {
        listArray = (E[]) new Object[size]; // Object 배열 생성 후 E 배열로 변환
        capacity = size;
        top = 0;
    }

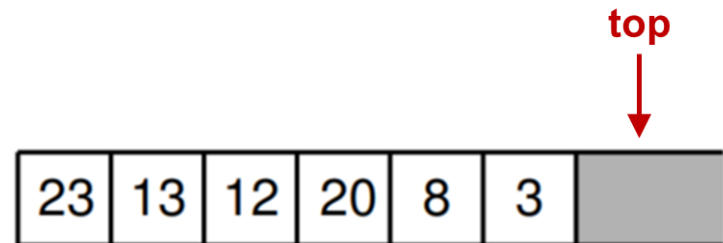
    public void push(E it) {
        if (top == capacity) throw new RuntimeException("Stack is full");
        listArray[top++] = it;
    }

    public E pop() {
        ...    // 각자 완성해보기
    }

    public E topValue() {
        ...    // 각자 완성해보기
    }
}
```

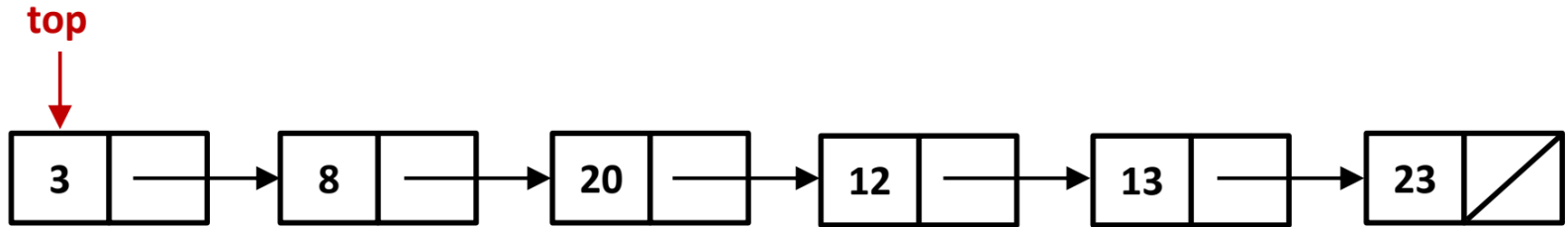
ArrayStack: 코드(정답)

```
public class ArrayStack<E> implements Stack<E> {  
    ...  
    public void push(E it) {  
        if (top == capacity) throw new RuntimeException("Stack is full");  
        listArray[top++] = it;  
    }  
    public E pop() {  
        if (top == 0) throw new RuntimeException("Stack is empty");  
        return listArray[--top]; // 실제 맨 위 원소는 top-1에 있음  
    }  
    public E topValue() {  
        if (top == 0) throw new RuntimeException("Stack is empty");  
        return listArray[top-1];  
    }  
}
```



LinkedStack: 구조

- 스택의 각 원소를 노드로 구성
 - 각 노드는 데이터(item)와 다음 노드 참조(next)를 가짐
- 스택의 top은 리스트의 맨 앞 노드를 가리킴



LinkedStack: 코드

```
public class LinkedStack<E> implements Stack<E> {
    private static class Node<E> {
        ...      // e.g., 연결 리스트 때 사용한 노드 클래스
    }

    private Node<E> top;  // 스택 맨 위 노드
    private int size;

    public LinkedStack() {
        top = null;
        size = 0;
    }

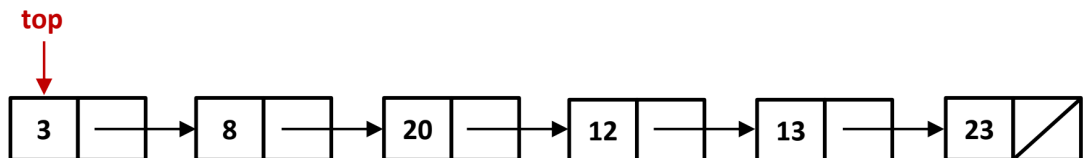
    public void push(E it) {
        top = new Node<>(it, top); // next node를 현재의 top으로
        size++;
    }

    public E pop() {
        ...      // 각자 완성해보기
    }

    public E topValue() {
        ...      // 각자 완성해보기
    }
}
```


LinkedList: 코드(정답)

```
public class LinkedList<E> implements Stack<E> {  
    ...  
    public void push(E it) {  
        top = new Node<>(it, top); // next node를 현재의 top으로  
        size++;  
    }  
    public E pop() {  
        if (top == null) throw new RuntimeException("Stack is empty");  
        E val = top.item;  
        top = top.next;  
        size--;  
        return val;  
    }  
    public E topValue() {  
        if (top == null) throw new RuntimeException("Stack is empty");  
        return top.item;  
    }  
}
```



Time Complexity (시간 복잡도)

- 일반적으로, 두 구현 방식 모두 모든 연산이 상수 시간 $O(1)$ 내 수행 가능

연산	ArrayStack	LinkedStack	참고사항
push	$O(1)$	$O(1)$	ArrayStack의 경우, 배열(listArray)이 가득차면 $O(n)$ 크기의 재할당 필요
pop	$O(1)$	$O(1)$	ArrayStack은 배열의 top 인덱스 감소, LinkedStack은 top 노드 제거
topValue	$O(1)$	$O(1)$	
length	$O(1)$	$O(1)$	ArrayStack은 top 인덱스 값, LinkedStack은 size 변수 유지
clear	$O(1)$	$O(1)$	

Space Complexity (공간 복잡도)

- **ArrayStack:** 배열 크기만큼의 공간만 필요
 - 단, 배열 크기 초과 시, **재할당 비용** 발생
- **LinkedStack:** 각 노드마다 포인터 공간이 추가적으로 필요

ArrayStack	LinkedStack
$O(n)$	$O(n)$ + 추가적인 메모리(포인터)

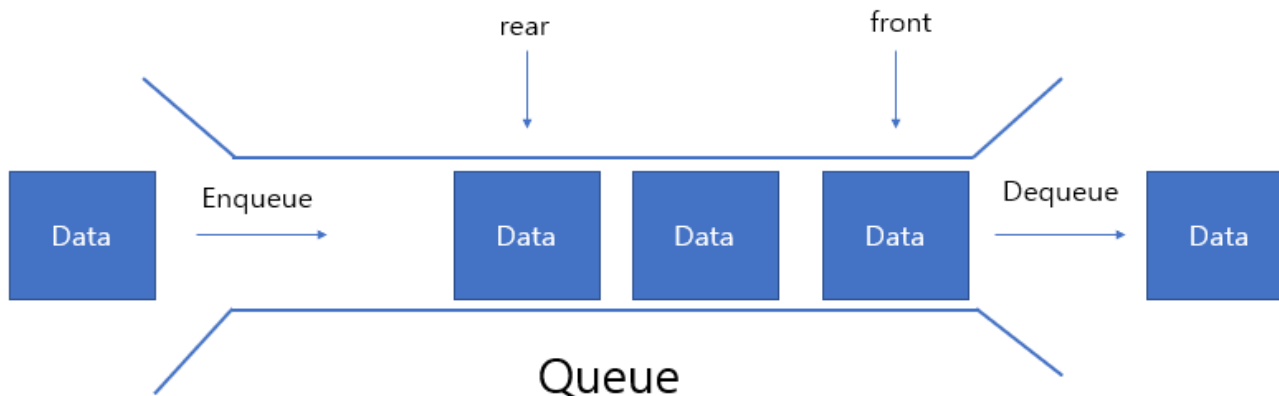
큐: 구조 및 주요 연산

- 구조

- 리스트 상에 **두 개의 끝**이 존재
 - **Rear (뒤):** 데이터를 삽입하는 쪽
 - **Front (앞):** 데이터를 삭제하는 쪽

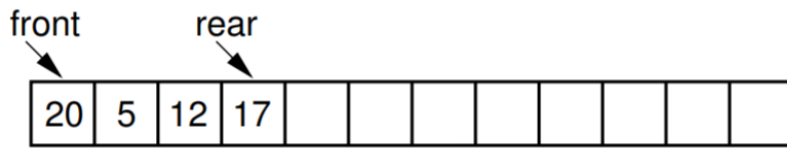
- 주요 연산

- **enqueue(item):** 큐의 뒤쪽에 원소(item)를 삽입
- **dequeue():** 큐의 앞쪽에서 원소를 꺼내어 제거
- **frontValue():** 큐의 맨 앞 원소를 제거하지 않고 확인 (리턴)

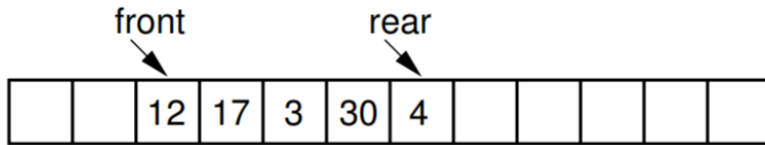


ArrayQueue: 구조

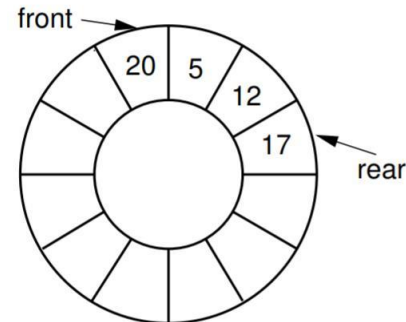
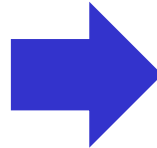
- 내부에 **고정된 크기**의 배열 존재
- 배열을 **원형 큐 (circular queue)** 구조로 구현
 - 공간 낭비 방지



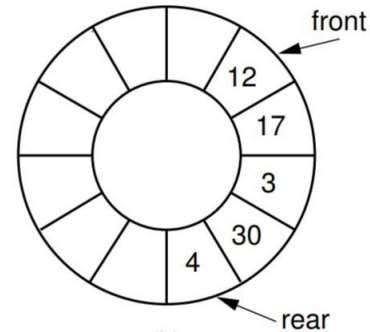
(a)



(b)



(a)



(b)

원형 큐 구조가 아닐 경우:
사용할수록 **배열의 한쪽 끝으로 점점 이동하게 됨**

원형 큐 구조

ArrayQueue: 코드

```
public class ArrayQueue<E> implements Queue<E> {
    private E[] listArray;    // 큐 원소 저장
    private int front;        // 삭제 위치
    private int rear;         // 삽입 위치
    private int size;         // 원소 개수
    private int capacity;     // 배열 용량

    public ArrayQueue(int capacity) {
        listArray = (E[]) new Object[capacity];
        this.capacity = capacity;
        front = 0; rear = 0; size = 0;
    }

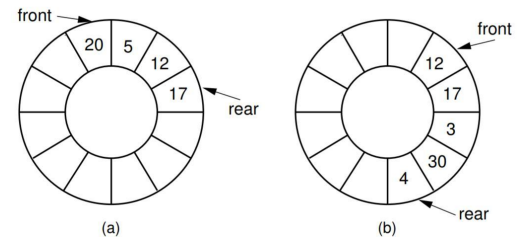
    public void enqueue(E it) {    // 큐에 새 item 삽입
        if (size == capacity) throw new RuntimeException("Queue is full");
        listArray[rear] = it;
        rear = (rear+1) % capacity;    // circular queue 방식으로 rear 인덱스 업데이트
        size++;
    }

    public E dequeue() {
        ...    // 각자 완성해보기
    }

    public E frontValue() {
        ...    // 각자 완성해보기
    }
}
```

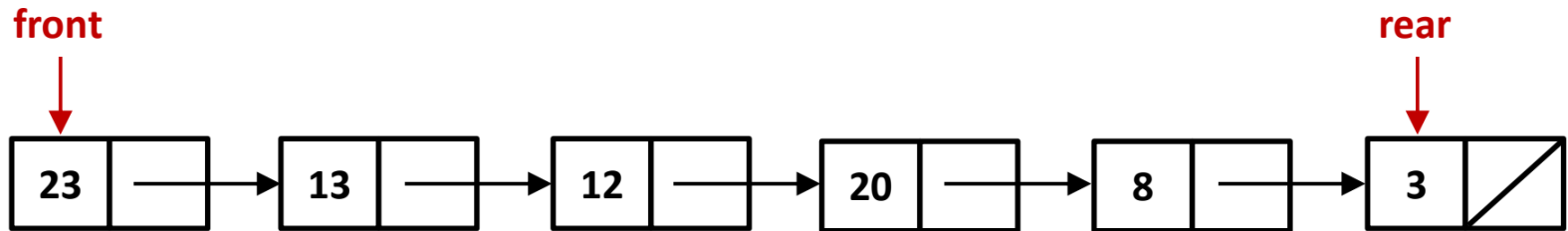
ArrayQueue: 코드(정답)

```
public class ArrayQueue<E> implements Queue<E> {  
    ...  
    public void enqueue(E it) { // 큐에 새 item 삽입  
        if (size == capacity) throw new RuntimeException("Queue is full");  
        listArray[rear] = it;  
        rear = (rear+1) % capacity; // circular queue 방식으로 rear 인덱스 업데이트  
        size++;  
    }  
    public E dequeue() {  
        if (size == 0) throw new RuntimeException("Queue is empty");  
        E val = listArray[front];  
        front = (front+1) % capacity;  
        size--;  
        return val;  
    }  
    public E frontValue() {  
        if (size == 0) throw new RuntimeException("Queue is empty");  
        return listArray[front];  
    }  
}
```



LinkedList: 구조

- 큐의 각 원소를 노드로 구성
 - 각 노드는 데이터(item)와 다음 노드 참조(next)를 가짐
- 큐의 rear 포인터에서 삽입, front 포인터에서 삭제 연산 수행



LinkedList: 코드

```
public class LinkedList<E> implements Queue<E> {
    private static class Node<E> {
        ...      // e.g., 연결 리스트 때 사용한 노드 클래스
    }

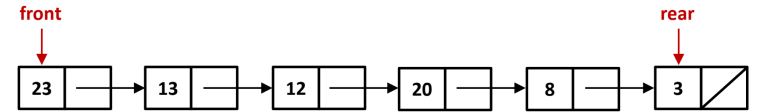
    private Node<E> front;    // 큐의 맨 앞 노드 (삭제 위치)
    private Node<E> rear;    // 큐의 맨 뒤 노드 (삽입 위치)
    private int size;

    public LinkedList() {
        front = rear = null;
        size = 0;
    }

    public void enqueue(E it) {    // 큐에 새 노드 삽입
        Node<E> node = new Node<>(it, null);
        if (rear == null) // 큐가 비어있을 때
            front = rear = node;
        else {
            rear.next = node;
            rear = node;    // rear를 새 노드로 업데이트
        }
        size++;
    }

    public E dequeue() {...}    // 각자 완성해보기
    public E frontValue() {...}    // 각자 완성해보기
}
```

LinkedList: 코드(정답)



```
public class LinkedList<E> implements Queue<E> {
    ...
    public void enqueue(E it) {        // 큐에 새 노드 삽입
        Node<E> node = new Node<>(it, null);
        if (rear == null) // 큐가 비어있을 때
            front = rear = node;
        else {
            rear.next = node;
            rear = node;        // rear를 새 노드로 업데이트
        }
        size++;
    }

    public E dequeue() {
        if (size == 0) throw new RuntimeException("Queue is empty");
        E val = front.item;
        front = front.next;
        if (front == null) rear = null; // 마지막 남은 노드를 삭제한 경우
        size--;
        return val;
    }

    public E frontValue() {
        if (size == 0) throw new RuntimeException("Queue is empty");
        return front.item;
    }
}
```

Time Complexity (시간 복잡도)

- 일반적으로, 두 구현 방식 모두 모든 연산이 상수 시간 $O(1)$ 내 수행 가능

연산	ArrayStack	LinkedStack	참고사항
enqueue	$O(1)$	$O(1)$	ArrayQueue의 경우, 배열(listArray)이 가득차면 $O(n)$ 크기의 재할당 필요
dequeue	$O(1)$	$O(1)$	삭제는 front 위치에서 수행
frontValue	$O(1)$	$O(1)$	
Length	$O(1)$	$O(1)$	
Clear	$O(1)$	$O(1)$	

Space Complexity (공간 복잡도)

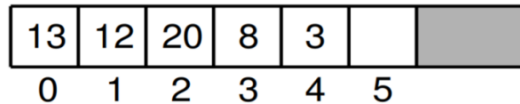
- **ArrayQueue:** 배열 크기만큼의 공간만 필요
 - 단, 배열 크기 초과 시, **재할당 비용** 발생
- **LinkedList:** 각 노드마다 포인터 공간이 추가적으로 필요

ArrayStack	LinkedList
$O(n)$	$O(n)$ + 추가적인 메모리(포인터)

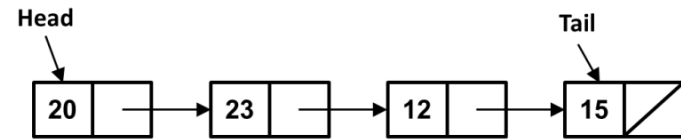
Doubly Linked List

One-way Linear Data Structure

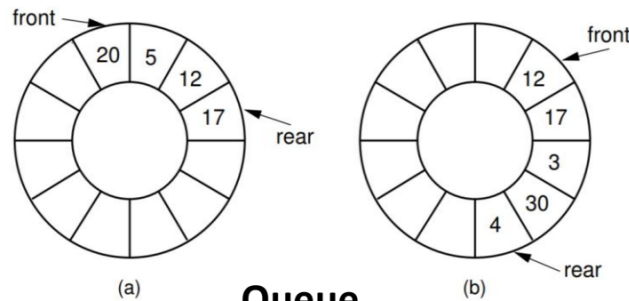
- 단방향 선형 구조: 배열, 연결리스트, 스택, 큐의 공통점
 - 선형 자료 구조: 데이터(원소)가 일렬로 배치되어 있고, 접근 순서가 일정한 구조
 - 단방향적 성격: 데이터 접근이 주로 한쪽 방향에서만 이루어짐



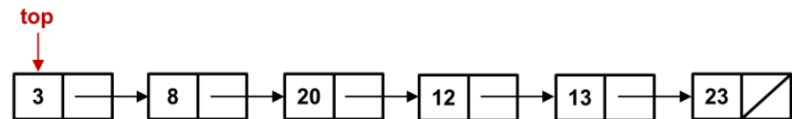
Array



Linked list



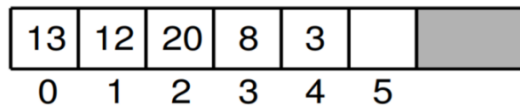
Queue



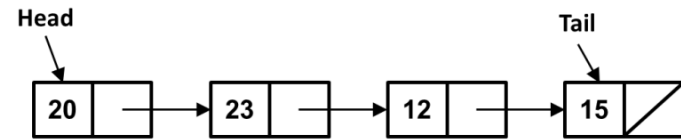
Stack

One-way (Uni-directional)

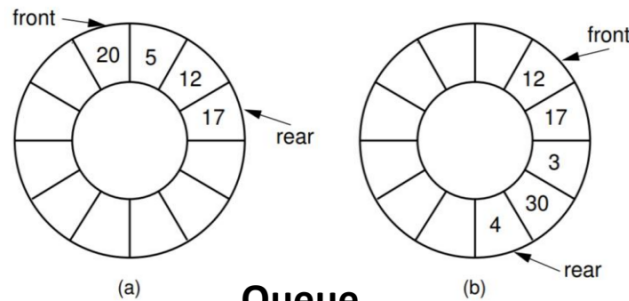
- 배열: 순차탐색, 삽입, 삭제 등의 연산이 단방향적으로 이뤄짐
- 연결 리스트: 구조적으로 단방향 (앞에서 뒤로만 이동 가능)
- 스택: 한쪽 끝에서만 접근 가능 (top에서만 삽입 및 삭제 연산 가능)
- 큐: 삽입에서부터 삭제까지의 흐름이 (앞에서 뒤로) 단방향



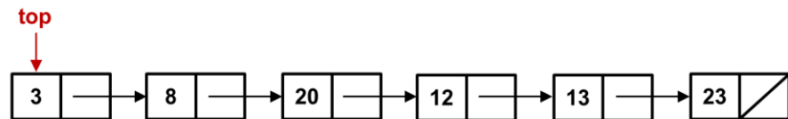
Array



Linked list

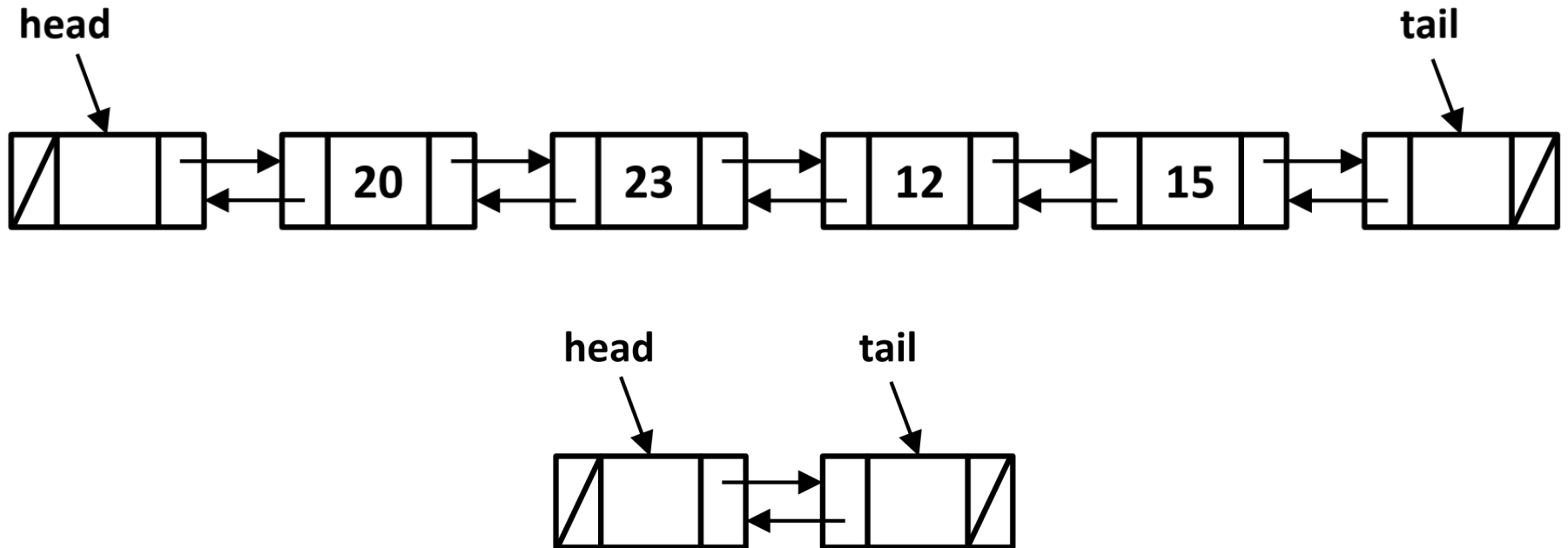


Queue



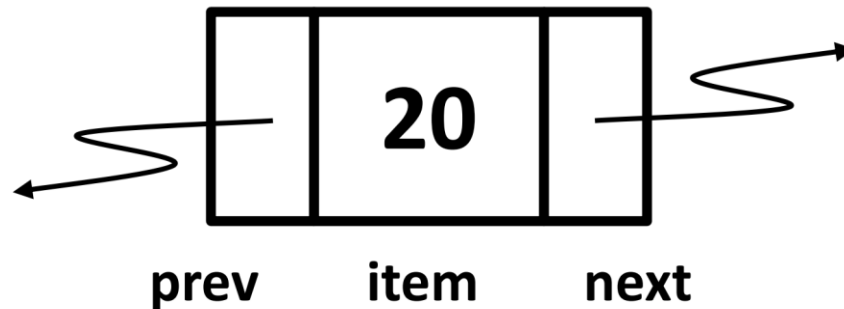
Stack

Doubly Linked List 이중 연결 리스트



Doubly Linked List

- 구조
 - 각 노드가 “이전 노드 (prev)”와 “다음 노드 (next)”에 대한 포인터를 모두 갖는 연결 리스트
- 특징
 - 양방향 탐색 가능
 - 삽입/삭제 연산 효율적
 - 메모리 사용량 증가

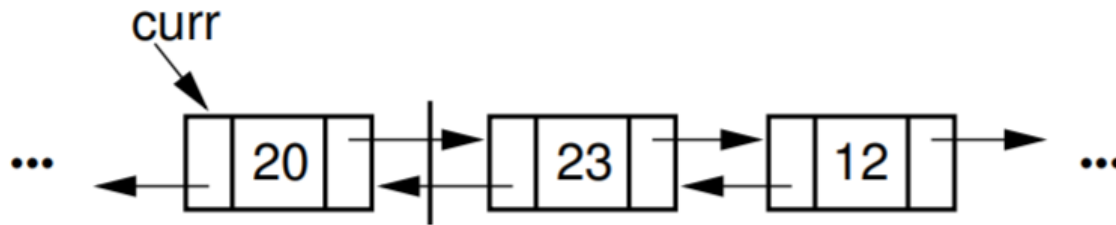


구현 예

```
public class DoublyLinkedList<E> {  
    /* DoublyLinkedList 클래스 내부에서만 접근 가능한 내부 클래스 */  
    private static class Node<E> {  
        E item;  
        Node<E> prev;    // 이전 노드 포인터  
        Node<E> next;    // 다음 노드 포인터  
  
        Node(E item) {    // 생성 시 item만 받음  
            this.item = item;  
            this.prev = null;  
            this.next = null  
        }  
    }  
    private Node<E> head;  
    private Node<E> tail;  
    private int size;  
  
    public DoublyLinkedList() {  
        head = null;  
        tail = null;  
        size = 0;  
    }  
    // Operations  
}
```

삽입

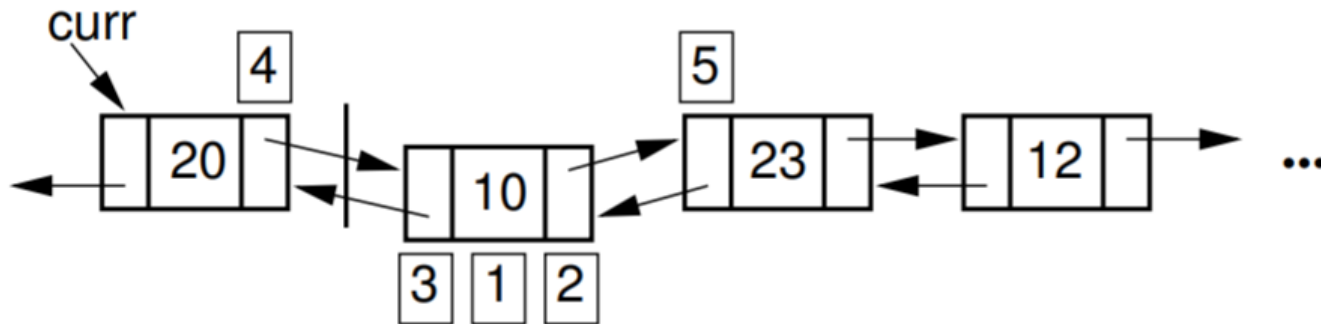
- 20과 23 사이에 10을 넣어야 하는 경우



Insert 10:

	10	
--	----	--

(a)



(b)

삽입: 구현

```
public void insert(int pos, E item) {  
    Node<E> newNode = new Node<>(item);  
    Node<E> curr = head;  
  
    for (int i=0; i<pos; i++)  
        curr = curr.next;  
  
    Node<E> prevNode = curr.prev;  
  
    newNode.next = curr;  
    newNode.prev = prevNode;  
    prevNode.next = newNode;  
    curr.prev = newNode;  
  
    size++;  
}
```

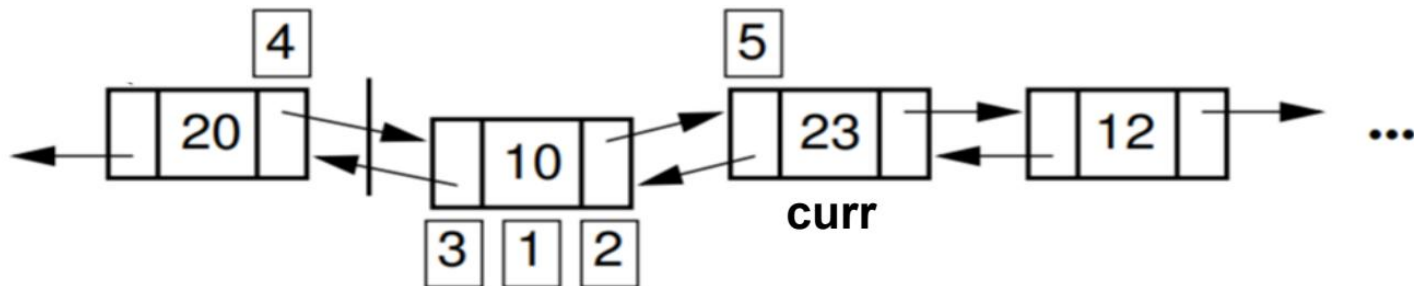
1

2

3

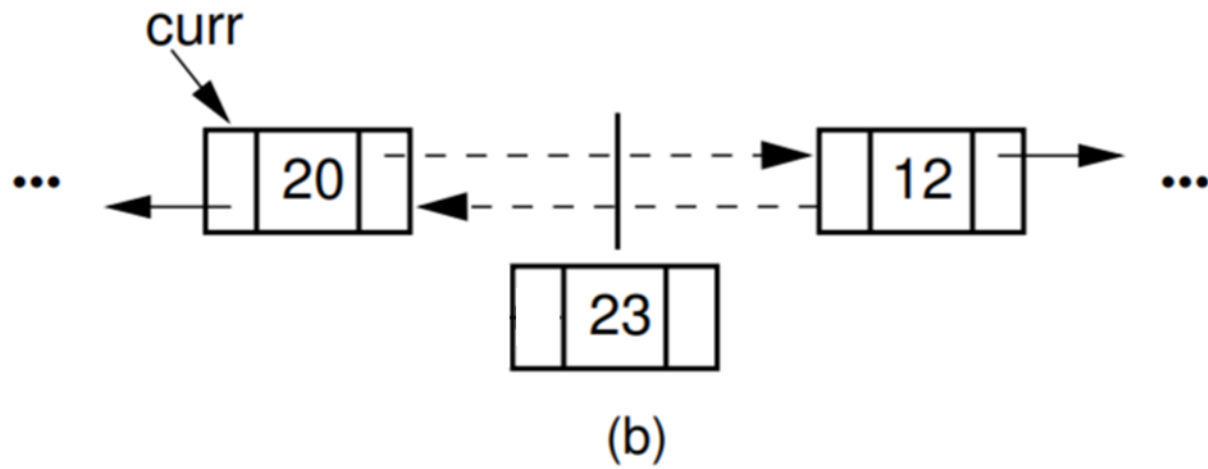
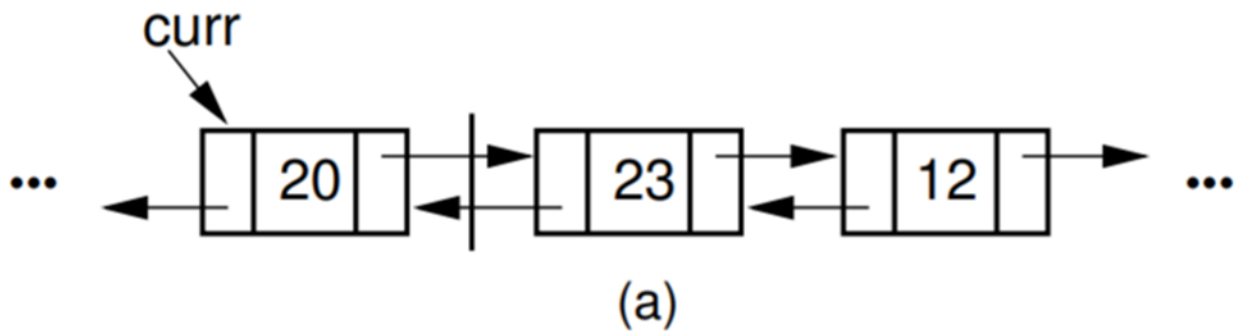
4

5



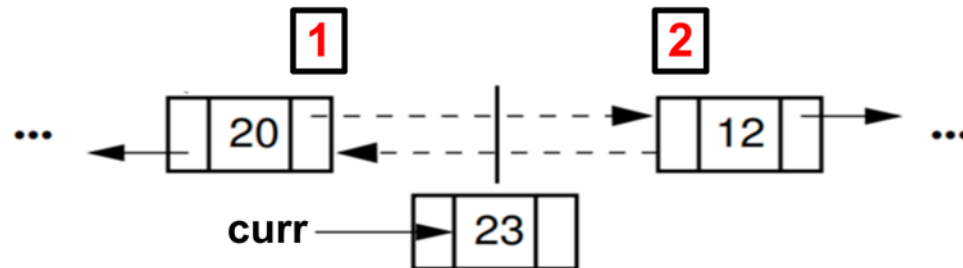
삭제

- 20과 12 사이에 있는 23을 지워야 하는 경우



삭제: 구현

```
public E delete(int pos) {  
    Node<E> curr = head;  
  
    for (int i=0; i<pos; i++)  
        curr = curr.next;  
  
    Node<E> prevNode = curr.prev;  
    Node<E> nextNode = curr.next;  
  
    prevNode.next = nextNode; 1  
    nextNode.prev = prevNode; 2  
  
    size--;  
    return curr.item;  
}
```



탐색: 구현

- 앞 → 뒤

```
public void traverseForward() {  
    Node<E> curr = head;  
    while (curr != null) {  
        System.out.print(curr.data + " ");  
        curr = curr.next;  
    }  
    System.out.println();  
}
```

- 뒤 → 앞

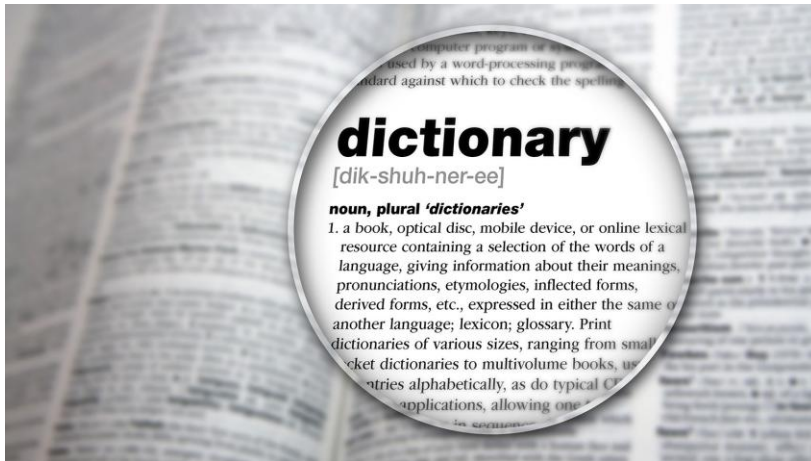
```
public void traverseBackward() {  
    ...  
}
```

main(): 이중연결리스트 사용

```
public static void main(String[] args) {  
    DoublyLinkedList<Integer> dll = new DoublyLinkedList<>();  
  
    dll.insert(0, 10);  
    dll.insert(1, 20);  
    dll.insert(2, 30); // [10 ↔ 20 ↔ 30]  
  
    dll.traverseForward();  
  
    dll.delete(1);  
    dll.traverseForward();  
}
```


Dictionary

Dictionary



SHARP

소프트 전자사전 **리얼리**



Dictionary: 정의 및 구조

- 정의
 - 데이터를 Key-Value 쌍으로 저장하는 선형 구조
 - 배열 또는 연결 리스트는 위치(인덱스)를 통해 원소를 찾는 반면,
 - 딕셔너리는 key를 통해 원하는 값(원소)을 빠르게 찾을 수 있음
(즉, “key 기반의 리스트”와 같음)

Dictionary: 정의 및 구조

- 정의
 - 데이터를 Key-Value 쌍으로 저장하는 선형 구조
 - 배열 또는 연결 리스트는 위치(인덱스)를 통해 원소를 찾는 반면,
 - 딕셔너리는 key를 통해 원하는 값(원소)을 빠르게 찾을 수 있음 (즉, “key 기반의 리스트”와 같음)
- 구조
 - 내부적으로는 배열 구조. (배열의) 각 칸에 key-value 쌍을 함께 저장
 - Value는 중복 가능하지만, key는 유일해야 함
 - Key를 해시함수(Hash function)를 통해 (배열의) 인덱스로 변환
 - Key 자체를 배열의 직접적인 인덱스로 사용할 수는 없기 때문

Dictionary: 정의 및 구조

- 정의

- 데이터를 **Key-Value 쌍으로 저장**하는 선형 구조
 - 배열 또는 연결 리스트는 위치(인덱스)를 통해 원소를 찾는 반면,
 - 딕셔너리는 key를 통해 원하는 값(원소)을 빠르게 찾을 수 있음 (즉, “**key 기반의 리스트**”와 같음)

- 구조

- 내부적으로는 배열 구조. (**배열의**) 각 칸에 **key-value 쌍을 함께 저장**
- Value는 중복 가능하지만, **key는 유일해야 함**
- Key를 **해시함수(Hash function)**를 통해 (**배열의**) 인덱스로 변환
 - Key 자체를 배열의 직접적인 인덱스로 사용할 수는 없기 때문

Key → [Hash Function] → Index

Index	0	1	2	3	4	5	6	7
Slot	null	(2025001, "Alice")	(2025017, "Cindy")	null	(2023049, "Jack")	null	null	(2022057, "John")

주요 연산 및 특징

- 주요 연산

- 삽입 (put): 새로운 key-value 쌍 추가
- 탐색 (get): key를 통해 value를 찾음
- 삭제 (remove): 특정 key-value 쌍 제거
- 수정 (put): 동일 key에 새로운 value 할당
- 키 존재 여부 확인 (contains): 딕셔너리 상에서 특정 key의 존재 여부 확인

- 특징

- 빠른 탐색

- 일반적인 리스트에서 특정 값을 찾을 땐, 기본적으로 순차 탐색 필요 ($O(n)$)
 - 하지만 딕셔너리에선 (key를 이용하므로) 평균 $O(1)$ 에 탐색 가능

- 순서 없음

- 일반적인 리스트는 구조적으로 앞에서 뒤 또는 뒤에서 앞으로의 순서가 존재
 - 하지만 딕셔너리는 순서를 유지하지 않음 ($\rightarrow$ 정렬을 지원하지 않음)

- 해시 충돌 (hash collision)

- 서로 다른 두 개 이상의 key가 해시 함수를 통해 같은 인덱스로 변환되는 상황
 - 삽입/삭제/탐색 연산 모두 최대 $O(n)$ 의 비용이 발생할 수 있음

구현: HashMap 구조 이용

```
import java.util.HashMap;

public static void main(String[] args) {
    HashMap<String, String> dict = new HashMap<>();

    dict.put("2025001", "Alice");    // 삽입
    dict.put("2025017", "Cindy");
    dict.put("2023049", "Jack");
    dict.put("2022057", "John");

    System.out.println("2025017: " + dict.get("2025017"));    // 탐색

    dict.put("2025001", "Bob");    // 수정
    dict.remove("2023049");    // 삭제

    if (dict.containsKey("2025005"))    // 키 존재 여부 확인
        System.out.println("Exists")
}
```

Look ahead

Tree